

Guide de Présentation Complet — IaC Chatbot

Tout comprendre pour répondre à TOUTES les questions du jury / de la société

PARTIE 1 — Vue d'ensemble

1.1 Le projet en 30 secondes (elevator pitch)

« J'ai développé une plateforme web qui permet à n'importe qui — même sans connaissances techniques — de créer de l'infrastructure informatique (machines virtuelles, conteneurs) en écrivant simplement sa demande en français ou en anglais dans un chatbot. Une IA 100% locale comprend la demande, génère le code Infrastructure as Code, et déploie réellement la ressource. Exemple : je tape "je veux une VM Ubuntu avec 2 CPU et 4 Go de RAM" → la VM est créée automatiquement dans VirtualBox. »

1.2 Le projet en 2 minutes (version détaillée)

1. **Problème** : déployer une VM ou un conteneur demande des experts coûteux (VMware, OpenShift, Terraform, YAML). Les demandes passent par des tickets, lentes et sources d'erreurs.
2. **Solution** : un chatbot qui transforme le langage naturel en code d'infrastructure, puis qui déploie réellement la ressource.
3. **Originalité** : l'IA est 100% locale (Ollama + Llama 3) — aucune donnée ne quitte l'entreprise, coût 0 €.
4. **Résultat** : une plateforme web complète — chat, génération de code, déploiement réel sur VirtualBox (VMs) et OpenShift (conteneurs), historique, dashboard admin, notifications temps réel.

1.3 Le problème et la solution

Le problème	La solution
Déployer une VM ou un conteneur demande des experts (VMware, OpenShift, Terraform, YAML...)	Un chatbot en langage naturel accessible à tous
Les déploiements manuels sont lents et sources d'erreurs	Automatisation complète : texte → code → déploiement
Les outils d'IA cloud coûtent cher et envoient les données à l'extérieur	IA 100% locale (Ollama), zéro coût, zéro fuite de données
Chaque plateforme a ses propres outils	Interface unique qui cache la complexité
Pas de traçabilité sur qui a demandé quoi	Historique complet en base de données

PARTIE 2 — Fonctionnement technique détaillé

2.1 Fonctionnement global (les 6 étapes)

Utilisateur (langage naturel)

[1] Chatbot Angular Interface web	[2] Backend Spring Boot API REST + JWT	[3] IA Ollama/llama3 Extraction des paramètres (type, plateforme, CPU, RAM...)
[6] Ressource créée VM VirtualBox ou conteneur OpenShift	[5] RealDeployExecutor Exécution réelle VBoxManage / oc	[4] Génération code IaC Templates Thymeleaf Terraform / YAML

1. L'utilisateur écrit sa demande dans le chat

2. Le backend reçoit le message (API sécurisée JWT)
3. L'IA locale (llama3) extrait les paramètres techniques en JSON
4. Thymeleaf génère le code IaC (Terraform pour VM, YAML pour OpenShift)
5. Le pipeline de déploiement exécute : VALIDATION → PLAN → APPLY → VERIFY
6. La ressource existe réellement (VM dans VirtualBox, pods sur OpenShift)

2.2 Zoom : comment l'IA extrait les paramètres

Question piège : « Comment l'IA sait-elle quoi extraire ? »

1. Le backend envoie à Ollama un **prompt système** qui décrit exactement le format JSON attendu : type de ressource (VM/CONTAINER), plateforme (VSPHERE/OPENSHIFT), osImage, cpu, ramGb, storageGb, replicas, containerImage, network.
2. Llama 3 répond avec du JSON.
3. LlmService parse le JSON. Si le parsing échoue → valeurs par défaut sûres (VM, VSPHERE, 2 CPU, 4 Go, 50 Go) — l'application ne plante jamais.
4. Température = 0.2 → réponses déterministes et stables (pas de créativité non désirée).

Pourquoi JSON et pas du texte libre ? Parce que le code doit être généré de manière 100% fiable : les paramètres structurés alimentent les templates.

2.3 Zoom : la génération du code IaC

- Les templates sont des fichiers versionnés dans `resources/templates/` :
 - `terraform/vmware-vm.tf` → VM VMware vSphere
 - `openshift/deployment.yaml + service.yaml + route.yaml` → conteneur OpenShift
 - `openshift/kubevirt-vm.yaml` → VM KubeVirt
- Thymeleaf en mode TEXT remplace les variables (`[[${cpu}]]`, `[[${ramGb}]]`...) par les valeurs extraites.
- **Avantage** : le code généré est toujours correct, standardisé, sans faute de syntaxe.
- Le code est **visible et copiable** par l'utilisateur avant déploiement (transparence).

2.4 Zoom : le pipeline de déploiement réel

Étape	VM (VirtualBox)	OpenShift (MicroShift)
VALIDATION	<code>VBoxManage --version</code> + vérification doublon	<code>oc apply --dry-run=server</code>
PLAN	Résumé des ressources (CPU, RAM, disque, OS)	Liste des ressources (<code>-o name</code>)
APPLY	<code>createvm</code> , <code>modifyvm</code> , <code>createmedium</code> , <code>storagectl</code> , <code>storageattach</code>	<code>oc apply -f manifest.yaml</code>
VERIFY	<code>showvminfo</code> (état de la VM)	<code>oc get</code> (ressources créées)
Annulation	<code>controlvm poweroff</code> + <code>unregistervm --delete</code>	<code>oc delete -f manifest.yaml</code>

Chaque étape est journalisée en base (`DeploymentLog`) et poussée en WebSocket.

2.5 Zoom : la sécurité

- **JWT** (JSON Web Token) : à la connexion, le serveur signe un token avec une clé secrète. Chaque requête suivante envoie ce token — le serveur vérifie la signature sans base de données.
- **BCrypt** : les mots de passe sont hashés (jamais stockés en clair).
- **Rôles** : USER (ses propres demandes) / ADMIN (tout + validation + quotas).
- **Aucun secret en dur** : tout passe par variables d'environnement.
- **CORS** : seul le frontend autorisé peut appeler l'API.

2.6 Zoom : le temps réel (WebSocket STOMP)

- Le déploiement peut prendre 30 secondes à plusieurs minutes.
- Sans WebSocket : l'utilisateur attend sans savoir ce qui se passe.
- Avec WebSocket : le serveur **pousse** chaque étape (« Validation... », « Création... », « Terminé ! ») vers le navigateur en direct.

2.7 Zoom : la base de données

- **H2** (dev) : base en mémoire, démarre instantanément, parfaite pour développer.
 - **MySQL** (prod) : base persistante, dans Docker.
 - Les entités : **User**, **InfrastructureRequest** (demande + code généré + statut), **DeploymentLog** (chaque étape de chaque déploiement).
-

PARTIE 3 — Pourquoi chaque technologie ? (LES QUESTIONS CLASSIQUES)

3.1 Pourquoi Angular pour le frontend ?

- Framework complet et structuré (standard entreprise)
- TypeScript = moins de bugs
- Composants réactifs (RxJS) parfaits pour un chat temps réel
- *Alternative écartée : React — Angular offre une structure plus cadrée pour un projet d'entreprise*

3.2 Pourquoi Spring Boot pour le backend ?

- Standard de l'industrie Java en entreprise
- Écosystème complet : Spring Security (JWT), Spring Data JPA (base), Spring AI (Ollama)
- Robuste, testable, documenté
- *Alternative écartée : Node.js/Express — moins adapté au typage fort et aux gros projets*

3.3 Pourquoi Ollama + Llama 3 et pas ChatGPT ?

- **100% local et offline** : aucune donnée ne quitte l'entreprise (confidentialité)
- **0 €** : pas d'abonnement API (OpenAI coûte 20-50\$/mois)
- Pas de quota, pas de limite de requêtes
- *C'est une exigence du cahier des charges : coût zéro*

3.4 Pourquoi Thymeleaf pour générer le code IaC ?

- Moteur de templates intégré à Spring Boot (zéro dépendance en plus)
- Mode TEXT : parfait pour générer du Terraform/YAML (pas seulement du HTML)
- Templates versionnés = standardisation et reproductibilité

3.5 Pourquoi VirtualBox au lieu de VMware vSphere ?

- vSphere nécessite un serveur vCenter (datacenter) — impossible sur un PC personnel
- VirtualBox : gratuit, open-source, fait la même chose (création/gestion de VMs)
- **Le code généré reste du Terraform vSphere** : sur une vraie infrastructure d'entreprise, il suffit de changer la cible
- Sur ma machine : exécution réelle via VBoxManage (VM créée avec les vraies specs)

3.6 Pourquoi MicroShift et pas OpenShift Local (CRC) ?

- OpenShift Local exige **Hyper-V**, absent de Windows 11 Home (limitation Microsoft)
- MicroShift = **vrai OpenShift officiel Red Hat**, version allégée, dans une VM VirtualBox

- Même API, même commande `oc`, Routes, pods — comportement identique à OpenShift
- *Seule limite : pas de KubeVirt (VMs dans OpenShift) — il faudrait un cluster plus puissant*

3.7 Pourquoi JWT pour la sécurité ?

- Stateless : pas de session serveur, parfait pour une API REST
- Standard de l'industrie, compatible avec tout
- Rôles USER/ADMIN intégrés (workflow d'approbation admin)

3.8 Pourquoi H2 en dev et MySQL en prod ?

- H2 : zéro installation, en mémoire, parfait pour développer et tester
- MySQL Community : gratuit, fiable, standard pour la production

3.9 Pourquoi WebSocket (STOMP) ?

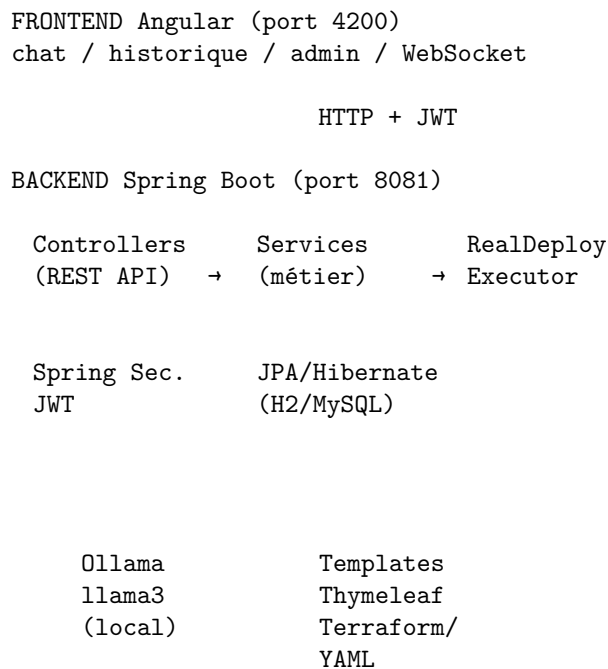
- Le déploiement prend du temps → l'utilisateur voit la progression **en temps réel**
- HTTP classique = le client doit demander ; WebSocket = le serveur pousse l'info

3.10 Pourquoi Docker Compose ?

- Reproductibilité : toute la stack (backend, frontend, MySQL) démarre avec une commande
- Isolation, portabilité, standard de l'industrie

PARTIE 4 — Architecture et code

4.1 Architecture technique (à savoir dessiner au tableau)



VirtualBox

OpenShift

(VBoxManage)	MicroShift 4.18
→ vraies VMs	(VM VirtualBox)
	→ vrais conteneurs

4.2 Structure du code (si on me demande de montrer le code)

```

iac-chatbot-backend/src/main/java/com/company/iacchatbot/
  controller/      → API REST (Auth, Chatbot, Deploy, History, Admin, User)
  service/         → logique métier
    LlmService.java      → dialogue avec Ollama
    IaCGeneratorService.java → génération du code (Thymeleaf)
    DeployService.java   → orchestration du déploiement
    RealDeployExecutor.java → EXÉCUTION RÉELLE (VBoxManage/oc)
    QuotaService.java    → quotas par utilisateur
    ProgressNotificationService → WebSocket temps réel
  model/           → entités (User, InfrastructureRequest, DeploymentLog)
  repository/      → accès base de données (JPA)
  security/        → JWT (filtre, génération, validation)
  config/          → configuration Spring

resources/templates/
  terraform/vmware-vm.tf      → template VM vSphere
  openshift/                  → templates YAML OpenShift/KubeVirt

```

Le fichier le plus important : `RealDeployExecutor.java` — c'est lui qui transforme la simulation en réalité (exécute les vraies commandes `VBoxManage` et `oc`).

4.3 Les requêtes HTTP de l'API

Endpoint	Description
POST /api/auth/register / login	Authentification JWT
POST /api/chatbot/process	Message → code IaC
GET /api/chatbot/requests	Historique des demandes
POST /api/deploy/{id}	Déployer (réel)
GET /api/deploy/{id}/status	Statut + logs du déploiement
DELETE /api/deploy/{id}	Annuler (suppression réelle)
GET /api/users (ADMIN)	Gestion des utilisateurs
GET /swagger-ui.html	Documentation interactive de l'API

PARTIE 5 — Infrastructure réelle (comment j'ai fait)

5.1 Le cluster OpenShift local (MicroShift)

Étape	Détail
VM	VirtualBox microshift : 2 vCPU, 4 Go RAM, 40 Go disque
OS	AlmaLinux 9 (compatible Red Hat) installé via image cloud + cloud-init
OpenShift	MicroShift 4.18 (dépôts officiels Red Hat)
Accès	Redirections NAT : API 16443→6443, Routes 9080→80 / 9443→443, SSH 2222→22

Étape	Détail
Auth	Compte de service <code>iac-backend</code> avec token
Pull secret	Compte Red Hat Developer (gratuit)

Pourquoi AlmaLinux et pas CentOS ? Les miroirs CentOS étaient bloqués par le réseau, AlmaLinux est 100% compatible Red Hat et accessible.

Pourquoi cloud-init ? VirtualBox ne sait pas installer EL9 automatiquement → image cloud pré-fabriquée + fichier de config cloud-init (utilisateur, SSH) injecté via un mini-CD (seed ISO).

5.2 Les VMs VirtualBox créées par le chatbot

- Nom automatique : `iac-vm-<id>` (id = numéro de la demande)
- Specs exactes extraites du message (CPU, RAM, disque)
- Type d'OS détecté automatiquement (ubuntu→Ubuntu_64, windows 2022→Windows2022_64...)
- **Note** : la VM est créée avec le matériel demandé. L'OS n'est pas installé (comme en entreprise : VMware crée la VM, l'OS vient d'un template séparé).

PARTIE 6 — Questions pièges et réponses

6.1 Questions sur le fonctionnement

« **Votre déploiement est-il vraiment réel ?** » → Oui. Démo : je crée une VM via le chatbot, elle apparaît dans VirtualBox avec les bonnes specs. Pour OpenShift : les pods tournent réellement (preuve avec `oc get pods`) et l'application est accessible via une Route HTTP.

« **Que se passe-t-il si l'IA comprend mal ?** » → L'IA retourne du JSON structuré avec des valeurs par défaut sûres. Le code généré est visible AVANT déploiement (l'utilisateur valide). Option : workflow d'approbation administrateur (PENDING_APPROVAL) avant tout déploiement.

« **Et si l'utilisateur demande quelque chose d'impossible ?** » → Combinaison invalide (ex: conteneur sur vSphere) → message de clarification (UC-07). Doublon de VM → refus avec message clair. Erreur d'exécution → statut FAILED + log.

6.2 Questions sur la sécurité

« **Comment sont protégés les mots de passe ?** » → Hashés avec BCrypt (jamais en clair, jamais réversibles).

« **Qui peut déployer ?** » → Tout utilisateur connecté pour ses propres demandes. L'admin peut tout voir/gérer. Option : approbation admin obligatoire avant déploiement.

« **Les secrets sont-ils dans le code ?** » → Non. Tout passe par variables d'environnement (JWT_SECRET, OC_TOKEN, mots de passe DB...).

6.3 Questions sur l'infrastructure

« **Ça marche sans internet ?** » → Oui pour l'IA (Ollama local) et la plateforme. Seul le téléchargement initial des images nécessite internet.

« **Comment ça passe à l'échelle ?** » → Backend stateless (JWT) → instances multiples possibles. Docker Compose → Kubernetes. La cible de déploiement peut passer de VirtualBox à un vrai vCenter en changeant la configuration, sans toucher au code.

« **Pourquoi pas de cloud (AWS/Azure) ?** » → Exigence du cahier des charges : plateformes cibles = VMware vSphere + OpenShift, clouds publics explicitement exclus. Coût 0 €.

« **Et KubeVirt (VMs sur OpenShift) ?** » → Le code YAML KubeVirt est généré correctement. L'exécution réelle demande un cluster avec virtualisation imbriquée (au-delà des 16 Go de RAM de ma machine). Sur une vraie infra, ça fonctionne avec le même code.

6.4 Questions sur la qualité

« **Combien de tests ?** » → 59 tests unitaires (100% passent) : génération IaC, extraction LLM, authentification, déploiement, quotas, notifications.

« **Quelles difficultés avez-vous rencontrées ?** » → (1) Windows 11 Home n'a pas Hyper-V → MicroShift dans VirtualBox. (2) Pas de vCenter sur PC → VirtualBox avec VBoxManage. (3) RAM limitée (16 Go) → optimisation des ressources des pods. (4) Miroirs CentOS bloqués → AlmaLinux. (5) VirtualBox ne sait pas installer EL9 automatiquement → cloud-init.

6.5 Questions sur le travail

« **Combien de temps ça a pris ?** » → 4 semaines : analyse, conception, développement, tests, documentation.

« **Qu'avez-vous appris ?** » → Full-stack (Angular/Spring Boot), intégration IA locale, IaC (Terraform/YAML), virtualisation (VirtualBox), conteneurisation (Docker/OpenShift), sécurité (JWT).

PARTIE 7 — Démonstration

7.1 Script de démo (5 minutes)

1. **Montrer** la page de login → connexion admin
2. **Taper** : « Je veux une VM Ubuntu avec 2 CPU et 4 Go de RAM et 20 Go de disque »
3. **Montrer** : paramètres extraits (2 CPU, 4 Go, 20 Go) + code Terraform généré
4. **Cliquer** Déployer → ouvrir VirtualBox → **la VM est là, en vrai**
5. **Taper** : « Déploie nginx avec 3 replicas sur OpenShift »
6. **Cliquer** Déployer → montrer `oc get pods` → **3 pods Running**
7. **Montrer** l'historique des demandes et les logs de déploiement
8. **Annuler** un déploiement → la VM disparaît de VirtualBox

Phrase de conclusion : « D'un texte en français à une infrastructure réelle, sans écrire une ligne de code — c'est ça, l'IaC pilotée par IA. »

7.2 Conseils pour la démo

- Garde Docker Desktop **fermé** (il mange de la RAM)
- Ferme les applications lourdes avant la démo
- Teste la démo **la veille** pour vérifier que tout marche
- Prépare les commandes de vérification dans un fichier texte

7.3 Vérifications rapides pendant la démo

Voir les VMs créées

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" list vms
```

Voir les pods OpenShift

```
cd "C:\Users\mouhi\Desktop\iac test\openshift"
```

```
.\oc.exe get pods --server=https://127.0.0.1:16443 --token=(Get-Content oc-token.txt) --insecure-skip-tls-
```

PARTIE 8 — Chiffres clés à retenir

Chiffre	Détail
0 €	coût total de la stack (tout open-source)
59	tests unitaires automatisés (100% passent)
6	étapes du pipeline (demande → ressource créée)
2	plateformes cibles (VMs + OpenShift)
100%	IA locale (aucune donnée ne sort)
4	étapes du déploiement réel (VALIDATION/PLAN/APPLY/VERIFY)
4.7 Go	taille du modèle IA (llama3, téléchargé une fois)
~30 sec	temps de déploiement d'une VM

PARTIE 9 — Glossaire rapide

Terme	Explication simple
IaC	Infrastructure décrite par du code (fichier texte), pas par des clics
LLM	Modèle de langage IA (comme ChatGPT, mais ici en local)
Ollama	Logiciel qui fait tourner des IA en local sur ton PC
Llama 3	Le modèle IA de Meta (gratuit, open-source)
Terraform	Outil qui décrit l'infrastructure en code (fichiers .tf)
VirtualBox	Logiciel gratuit pour créer des machines virtuelles
VBoxManage	La “télécommande” de VirtualBox en ligne de commande
OpenShift	Plateforme Red Hat pour gérer des conteneurs
MicroShift	Version légère d'OpenShift pour petits serveurs
oc	La “télécommande” d'OpenShift en ligne de commande
KubeVirt	Extension pour faire des VMs dans OpenShift
JWT	Token de sécurité pour prouver qui tu es
WebSocket	Connexion permanente entre navigateur et serveur (temps réel)
Thymeleaf	Moteur qui remplit des templates avec des variables
Spring Boot	Framework Java pour créer des applications web
Angular	Framework JavaScript pour créer des interfaces web